# Da Semana 4 à Semana 5

### Conectando services, signals e RxJS a formulários, rotas, pipes, diretivas e APIs

Desenvolvimento Front-End [Angular] T1 | Módulo 2 | Semana 5 | Este material não traz conteúdo novo: mostra, ponto a ponto, como o que você já praticou (**Módulo 1** e **Semanas 3 e 4**) é a **base literal** do que vamos fazer na **Semana 5**. A ideia é simples: **nada aqui é assunto desconhecido, é o mesmo conhecimento crescendo.**

---

### Como ler este material

Cada seção segue o mesmo formato: primeiro, **o que você já sabe** (com o código como apareceu antes); depois, **o que a Semana 5 acrescenta**; por fim, uma comparação lado a lado entre a versão "de antes" e a "de agora" do mesmo problema.

> **O fio condutor: o projeto que cresceu**
> Na Semana 3 você criou a **biblioteca-web**; na Semana 4 ela ganhou **service, signals, Observables, HTTP e Git em equipe**. Agora ela ganha **entrada de dados** (formulários), **várias páginas** (rotas), **formatação e reuso** (pipes e diretivas) e **dados de fora** (APIs). **Nada aqui é assunto desconhecido: é o mesmo conhecimento crescendo.**

| Semana | O que você construiu | Ferramenta usada |
|---|---|---|
| Módulo 1 | JavaScript no navegador e no Node: DOM, `fetch`, `async/await`, `localStorage`, regex | `document.querySelector`, `fetch`, `JSON.parse`, `replace(/\D/g, '')` |
| 03 e 04 | Componentes, `@for/@if`, bindings, `@Input/@Output`; service, DI, signals, Observables, `HttpClient`, GitFlow | `inject()`, `signal`, `input()`, `subscribe`, `retry`, `finalize` |
| 05 | A mesma biblioteca-web com formulários, rotas, pipes, diretivas e APIs | **Assunto desta semana:** `FormGroup`, `provideRouter`, `@Pipe`, `@Directive`, `HttpClient` |

## CONEXÃO 1 - Do estado em signals e da mensagem na tela aos formulários

### O que você já sabe (Semanas 3 e 4 e Módulo 1)

Na Semana 4 você guardou o estado de tela em **signals** (`carregando`, `erro`) e viu que, em aplicações zoneless, a tela só atualiza quando um signal muda. No Módulo 1, o formulário era lido com `document.querySelector(...).value`.

### O que a Semana 5 acrescenta

- `ngModel` e `FormControl` guardam o **valor e o estado** (válido, tocado, alterado) de cada campo.
- As mensagens de sucesso e de erro continuam em **signals**.
- O dado válido segue para o **service** (`adicionar()`), como na Semana 4.

## Comparando o código: o mesmo problema, antes e depois

**MÓDULO 1 - ler o campo na mão**

```
const nome = document.querySelector('#nome').value;
if (nome.length < 3) {
  erro.textContent = 'Mínimo 3 letras';
}
```

**SEMANA 5 - Reactive Forms**

```
form = this.fb.group({
  nome: ['', [Validators.required,
Validators.minLength(3)]],
});
// template: [formGroup]="form"
formControlName="nome"
```

**O que realmente mudou?**

O objetivo é o mesmo (validar o que o usuário digitou). Muda **quem acompanha o estado**: o Angular, em vez de código manual no DOM.

# CONEXÃO 2 - Do service, da DI e do inject() ao Router, FormBuilder e HttpClient

## O que você já sabe (Semanas 3 e 4 e Módulo 1)

Na Semana 4 você aprendeu que **não se cria** o que se precisa: **pede-se** ao Angular com `inject()` (nada de `new`) e recebe-se a **mesma instância**.

## O que a Semana 5 acrescenta

- `inject(NonNullableFormBuilder)`, `inject(Router)` e `inject(HttpClient)` seguem **exatamente** a mesma ideia.
- O `HttpClient` fica **dentro de um service** (`CepService`, `CatalogoService`), e o componente só usa o service.
- `provideRouter` e `provideHttpClient` são os **providers** que ensinam o Angular a entregar essas dependências.

## Comparando o código: o mesmo problema, antes e depois

**SEMANA 4 - um service seu**

```
private livroService = inject(LivroService);
this.livroService.emprestar(id);
```

**SEMANA 5 - serviços do Angular**

```
private router = inject(Router);
private http = inject(HttpClient);
this.router.navigate(['/livros', id]);
```

> **O que realmente mudou?**
>
> A mecânica não muda. A diferença é que agora a dependência **já vem pronta do Angular**, e você só precisa registrar o provider certo (`provideRouter`, `provideHttpClient`).

# CONEXÃO 3 - Do input()/output() e do model() aos parâmetros de rota e às diretivas

## O que você já sabe (Semanas 3 e 4 e Módulo 1)

Na Semana 4 você declarou entradas com `input()` e `input.required()` e viu o pai enviar `[livro]="x"` ao filho.

## O que a Semana 5 acrescenta

- Com `withComponentInputBinding()`, o **parâmetro da URL** ( `:id` ) chega ao componente como um `input()`.
- Diretivas também recebem dados com `input()` ( `[appDestaque]="'#c8e6c9'"` ).
- Todo parâmetro de rota chega como **texto**: converta com `Number()`.

## Comparando o código: o mesmo problema, antes e depois

**SEMANA 4 - pai envia ao filho**

```
livro = input.required<Livro>();
// pai: <app-card-livro [livro]="livro" />
```

**SEMANA 5 - a URL envia ao componente**

```
id = input.required<string>();
livro = computed(() => lista().find(l => l.id ===
Number(this.id())));
// URL: /livros/3
```

> **O que realmente mudou?**
> O componente **continua recebendo dados por** `input()`. Muda quem envia: o roteador, a partir da URL.

# CONEXÃO 4 - Dos Observables e do HttpClient aos valueChanges, ao CEP e aos interceptors

## O que você já sabe (Semanas 3 e 4 e Módulo 1)

Na Semana 4 você viu que um Observable é **preguiçoso**, só executa com `subscribe`, e que `retry`, `catchError` e `finalize` tratam falhas. Também aprendeu a **cancelar** inscrições (`takeUntilDestroyed`).

## O que a Semana 5 acrescenta

- `valueChanges` do campo CEP é um **Observable** (`filter` + `takeUntilDestroyed`) que dispara a consulta.
- `http.get<T>()` devolve um Observable; os **3 estados** da tela usam `finalize`.
- Um **interceptor** é uma função que passa por toda requisição (log, cabeçalhos, token).

## Comparando o código: o mesmo problema, antes e depois

**MÓDULO 1 - fetch e await**

```
const r = await
fetch('https://viacep.com.br/ws/88010400/json/');
const endereco = await r.json();
```

**SEMANA 5 - HttpClient + RxJS**

```
this.cepService.buscar(cep)
  .pipe(finalize(() => this.carregando.set(false)))
  .subscribe({ next: (e) => this.form.patchValue(e)
});
```

**O que realmente mudou?**

É a mesma conversa com a API, agora com **Observable**, tipagem, tratamento de erro padronizado e um ponto central para interceptar tudo.

# CONEXÃO 5 - Do Módulo 1 (regex, DOM e localStorage) às diretivas, pipes e guards

## O que você já sabe (Semanas 3 e 4 e Módulo 1)

No Módulo 1 você usou **expressões regulares** para limpar texto, escutou eventos do DOM com `addEventListener` e formatou datas e valores com funções manuais.

## O que a Semana 5 acrescenta

- **Diretivas** escutam eventos do elemento e mudam o estilo, como um `addEventListener` reutilizável ( `appDestaque` , `appSomenteNumeros` com `replace(/\D/g, '')` ).
- **Pipes** formatam valores só na tela ( `currency` , `date` ), no lugar de funções de formatação espalhadas.
- Um **guard** substitui o `window.confirm` solto na saída da página.

## Comparando o código: o mesmo problema, antes e depois

**MÓDULO 1 - formatar na mão**

```
const preco = 'R$ ' + valor.toFixed(2).replace('.',
',');
input.addEventListener('input', () => {
  input.value = input.value.replace(/\D/g, '');
});
```

**SEMANA 5 - pipe e diretiva**

```
{{ valor | currency: 'BRL' }}
<input appSomenteNumeros />
```

> **O que realmente mudou?**
>
> As ideias já eram suas (regex, eventos, formatação). Agora elas viram **peças reutilizáveis** do framework.

# CONEXÃO 6 - Do GitFlow à entrega do projeto da semana

## O que você já sabe (Semanas 3 e 4 e Módulo 1)

Na Semana 4 você praticou **branches** `feature/*`, Pull Request e Conventional Commits.

## O que a Semana 5 acrescenta

- Cada parte da semana pode ser uma `feature` (`feature/formularios`, `feature/rotas`, `feature/apis`).
- Mensagens como `feat(forms): cadastro de livro com validação` e `feat(http): busca de CEP`.
- Mantenha o repositório atualizado: é a base das próximas semanas.

## Comparando o código: o mesmo problema, antes e depois

**SEMANA 4**

```
git switch -c feature/busca develop
git commit -m "feat(busca): filtro por titulo"
```

**SEMANA 5**

```
git switch -c feature/formularios develop
git commit -m "feat(forms): cadastro de livro"
```

> **O que realmente mudou?**
>
> O fluxo é o mesmo. O que muda é o **conteúdo** que você versiona.